Intelligent Expense Tracker

An Automated Platform for Expense Capture, Collaborative Spending, and Financial Intelligence

Software Engineering - Semester Project Proposal

Team Members

Aashi Tyagi (1024170263) Sarwangini Hamal (1024170262)

Divyansh Kumar (1024170267) Gauransh Arora (1024170264)

Submitted to: Nisha Thakur

1. Introduction

Every digital payment already carries the information needed to understand it: a UPI transaction has a merchant, an amount, and a time; a printed bill lists individual items and prices; a bank statement records a running history of debits and credits. In practice, however, this information stays scattered across screenshots, paper receipts, and PDF statements, and none of it is usable until a person manually retypes it into an expense-tracking app.

This project proposes a system built around a simple idea: instead of asking users to record their expenses, the application should read the financial documents users already generate and turn them into a single, structured financial history. Expense splitting, analytics, and financial insights are then built on top of this history rather than on manually typed entries. The project is therefore not simply an expense tracker or a bill-splitting tool, but an automated financial record and analysis system, of which splitting and analytics are two of several components.

2. Problem Statement

Financial information is generated constantly, but it is not structured or easily usable. This produces several concrete problems:

- **Fragmented financial information.** Receipts contain merchant, item, and price detail; UPI screenshots contain transaction detail; bank statements contain a running transaction history. None of these sources are connected, so a user's complete spending picture never exists in one place.
- **Repeated manual effort.** Even though this information already exists digitally, users must retype it into a tracker, which is slow enough that most people stop doing it consistently.
- **Limited understanding of spending.** Most expense trackers can answer "how much did I spend", but not "where did it go", "why did it increase", or "what is recurring".
- **No decision support.** Recorded expenses are rarely used to evaluate a planned purchase, and shared expenses add a further layer of complexity because the person who pays and the people who benefit are often different.

The proposed system addresses this by automating the capture step and then using the resulting structured history to answer these harder questions, rather than stopping at transaction logging.

3. Objectives

Primary goal. To design and build a platform that automatically converts bills, UPI screenshots, and bank statements into a structured, verified financial history, and uses that history for collaborative expense management and spending intelligence.

Specific objectives:

- Automate expense capture from bills, receipts, UPI screenshots, and bank statements using OCR.
- Convert extracted information into structured, categorized transactions, including merchant, amount, date, and individual items, with user verification before storage.
- Support collaborative expense management through equal, percentage, and custom splitting, with debt tracking between users.
- Provide multi-dimensional spending analytics across categories, merchants, items, and time periods.
- Detect recurring expenses and unusual spending patterns from transaction history.
- Support spending decisions through purchase-readiness analysis and wishlist tracking.
- Enable natural-language interaction with the user's own financial history.

4. Proposed Solution

At a high level, the system takes any financial document a user already has, such as a bill, a UPI payment screenshot, or a bank statement, and turns it into a verified transaction. The user shares or uploads the document; the system extracts and normalizes the relevant fields and suggests a category; the user confirms or corrects this before it is saved. Once confirmed, the transaction becomes part of the user's financial history, which is then used by separate modules for collaborative splitting, analytics, recurring and unusual spending detection, purchase-readiness analysis, and natural-language queries.

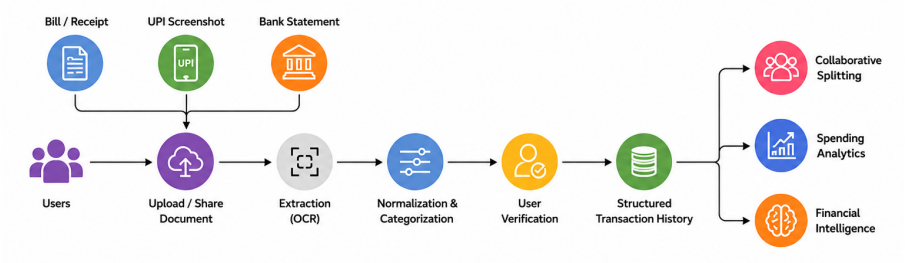

Figure 1: Overall System Flow

This flow keeps the user in control of what enters their financial record, while shifting the repetitive work of data entry onto the system.

5. System Architecture and Methodology

5.1 System Architecture

The system follows a three-tier architecture: a Flutter mobile frontend, a FastAPI backend exposing REST APIs, and a PostgreSQL database for persistent storage.

The backend is organized into an extraction layer for OCR and document parsing, a processing layer for categorization and verification, and an intelligence layer for splitting, analytics, pattern detection, and queries. These components read from and write to a shared PostgreSQL schema covering users, transactions, items, categories, splits, and wishlist entries.

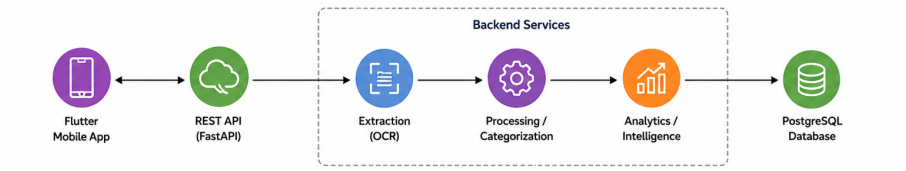

Figure 2: System Architecture

5.2 Automated Expense Extraction and Verification

The core extraction workflow is:

1. A user uploads a bill or shares a UPI payment screenshot.
2. OCR extracts raw text from the image.
3. The processing layer identifies merchant, amount, date, items, quantities, and taxes from the extracted text.
4. Transactions and individual items are categorized using merchant information, keywords, and lightweight classification.
5. The extracted information, along with a confidence score, is shown to the user for verification.
6. The confirmed transaction is stored in PostgreSQL and becomes available to every downstream module.

The system does not treat OCR output as final. Extracted data is always presented for user confirmation before it becomes part of the financial history, which limits how far an extraction error can propagate.

5.3 Collaborative Expense Management

Shared expenses such as meals, trips, hostel costs, or group purchases can be split between participants using equal, percentage-based, or custom amounts. The system tracks who paid, who owes what, and maintains a running settlement history between users, so debts do not need to be tracked manually outside the app.

5.4 Expense Tracking and Spending Analytics

The application maintains a chronological expense timeline containing merchant, amount, category, items, and the originating bill or screenshot. The analytics layer builds on this timeline to provide category-wise and monthly spending, merchant-level and item-level analysis, comparisons with previous months, and "where did my money go" summaries.

6. Financial Intelligence

This section covers the modules that use the stored transaction history to go beyond recording expenses and actually derive information from them. These modules are the main differentiator of the project over a standard tracker or splitting app.

6.1 Unusual Spending Detection

Historical spending per category is used to establish a normal range for the user. Transactions or category totals that deviate noticeably from this range are flagged for the user's attention. This is a historical-pattern-based check rather than a trained anomaly-detection model.

6.2 Recurring Expense and Subscription Detection

Transactions with similar merchants, amounts, and regular time intervals are identified as likely recurring expenses or subscriptions. This is inferred purely from transaction history rather than from direct access to UPI mandates or bank-level subscription data.

6.3 Purchase Readiness and Wishlist Analysis

Users can add a planned purchase to a wishlist along with its expected price. The system evaluates the purchase against current spending, monthly average spending, recurring payments, historical trends, savings goals, and discretionary budget, and returns a simple verdict: **Comfortable**, **Possible**, or **Not recommended currently**.

The system can also compare current-month spending with the user's historical average to identify whether the user currently has additional discretionary room for a wishlist purchase. This is intended as a data-driven spending indicator, not professional financial advice.

6.4 Ask Your Expenses

An "Ask Your Expenses" interface lets users query their own financial history in natural language instead of navigating separate dashboards.

Example questions include:

- "How much did I spend on food this month?"
- "What are my recurring monthly expenses?"
- "How much does Rahul owe me?"
- "Where did most of my money go this month?"

Supported questions are mapped to structured database queries or analytical operations and executed only against the user's own data.

6.5 Bank Statement Processing

Supported PDF or CSV bank statements can be uploaded and converted into structured transactions, which are normalized, categorized, and merged with existing expenses after user verification. Real-time bank synchronization is outside the initial scope, since it requires additional financial ecosystem access and consent mechanisms.

7. Scope and MVP

7.1 MVP Scope

The MVP includes bill and UPI screenshot extraction with user verification, expense categorization and history, collaborative splitting, spending analytics, recurring-expense detection, unusual spending detection, wishlist and purchase-readiness analysis, the Ask Your Expenses query interface, and supported bank-statement processing.

7.2 Out of Scope

Real-time bank and UPI account synchronization, direct control or blocking of third-party payments, and professional financial advisory or investment-recommendation features are outside the MVP. These require regulatory and financial ecosystem access beyond a semester project and are noted as future extensions instead.

8. Testing and Evaluation

Success will be measured against four concrete evaluation areas, chosen to match the project's central claims:

- **OCR extraction accuracy.** Extracted merchant, amount, date, and item fields will be compared against manually verified ground truth on a sample set of bills and screenshots.
- **Categorization accuracy.** The proportion of transactions assigned to the correct category, checked against user corrections.
- **Time saved.** The time taken to record an expense via upload and verification will be compared against fully manual entry.

- **Query correctness.** A fixed set of natural-language questions will be run against known data, and returned answers checked against the underlying database calculations.

Unit tests will cover backend services and transaction-processing logic, and integration tests will verify the complete document-to-database workflow before user testing begins.

9. Technology Stack and Resources

Technology stack. Flutter will provide the cross-platform mobile interface, covering authentication, screenshot sharing, image upload, expense verification, dashboards, and financial queries. FastAPI will expose REST APIs for authentication, transaction processing, splitting, analytics, and financial queries. PostgreSQL will store users, transactions, items, categories, splits, and wishlist data. Git and GitHub will be used for version control.

Human resources. The team will divide responsibilities across mobile development, backend development, database design, OCR and data processing, and testing.

Budget. No external funding is required for the prototype. OCR, PostgreSQL, and the backend can be implemented using free or open-source resources, with free-tier deployment for the staging build. Paid AI services and production financial integrations remain optional future requirements.

10. Timeline

The project will be developed incrementally, with the automated capture workflow completed before collaborative and intelligence features.

Weeks	Phase	Deliverable
1–2	Planning and Design: requirements, architecture, UI	ER diagram
3–4	Core Application: authentication, transaction model, manual entry	Mobile shell + backend
5–6	Automated Extraction: OCR, bill and UPI screenshot processing	Expense extraction module
7–8	Expense Management: categories, splitting, transaction history	Core expense tracker
9–10	Analytics: dashboards, category, merchant, and item spending	Analytics dashboard
11–12	Financial Intelligence: recurring and unusual spending detection, wishlist, and queries	Intelligence modules
13	Testing and Integration: unit, integration, and user testing	Integrated build
14	**Buffer**: bug fixing and optimization	Stabilised build
15	Deployment, documentation, and demonstration	Final application + report

11. Risks and Future Extensions

11.1 Risks and Mitigation

- **OCR inaccuracies**, especially for low-quality or unusual receipt formats, are mitigated through confidence scores and mandatory user verification before storage.
- **Inconsistent receipt and screenshot layouts** across merchants are mitigated by supporting common formats first and providing a manual-correction fallback.
- **Incorrect categorization** affecting analytics is mitigated through user correction, which is fed back into classification rules.
- **Financial data sensitivity** is mitigated through authentication, access control, secure transport, and data minimisation.

- **Scope creep**, given the number of modules, is mitigated through an MVP-first approach with clearly defined out-of-scope features.

11.2 Future Extensions

Future directions include consent-based bank and UPI account aggregation, more advanced anomaly detection, household or shared-family expense management, and more sophisticated budgeting and financial recommendations, once sufficient real usage data exists to support them responsibly.

A future parent dashboard could allow parents to define spending limits, monitor a child's spending, and receive alerts when a child approaches a budget threshold. However, directly blocking payments through third-party UPI applications is outside the core scope.

12. Summary

This project proposes the Intelligent Expense Tracker, a system that automates the capture of financial information from bills, UPI screenshots, and bank statements, and turns it into a verified, structured financial history. The central objective is to transform fragmented and unstructured transaction information into a structured financial history that can be automatically analyzed to help users understand and make better everyday spending decisions through collaborative splitting, spending analytics, recurring and unusual spending detection, purchase-readiness analysis, and natural-language queries, built on Flutter, FastAPI, and PostgreSQL.